Transmission des données

Université de Poitiers
P. Meseure
(Philippe.Meseure@univ-poitiers.fr)

Problématique

- Cycle de vie des données
  - Acquisition des données par chaque capteur
  - Stockage local impossible ou très limité
  - Centralisation des données (serveur) pour archivage et exploitation : analyse, visualisation, etc.
- Protocole pour transmettre des données
  Idéalement :
  - Protocole léger en taille des données
  - Protocole peu gourmand en énergie (attente)
- Ici, présentation de deux protocoles assez opposés
  - Service web (HTTP+REST) : protocoles classiques en informatique mais plutôt gourmands
  - MQTT : protocole plus léger, mais moins poussé

Approche « Services Web »

Protocole « HTTP »

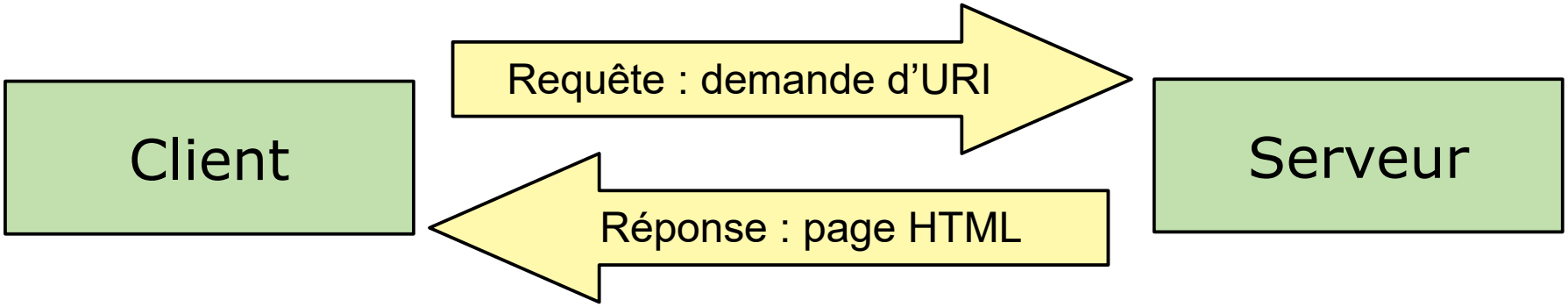
HTTP

- Apparition du protocole HTTP au milieu des années 90
  - Protocole support du Web
  - Permet de « transporter » des données diverses (type MIME)
    - Texte formaté (HTML + présentation CSS)
    - Images
    - Vidéo
    - Puis, avec le Web 2.0, des données pures (XML, JSON)

- HTTP = HyperText Transfert Protocol
  Un protocole pour transmettre des données (sous forme texte)

HTTP : aspects techniques

- Protocole basé sur TCP
- But : Dialoguer avec un serveur de ressources
- Modèle client/serveur
  - Requête HTTP : adressée par le client vers le serveur
  - Réponse HTTP : envoyée par le serveur au client



Requête HTTP

- Format d'URI (Universal Resource Identifier)
  - Protocole:\\nom_ou_ip_serveur:port\chemin\nom_ressource
    Exemple : http:\\univ-poitiers:80\formation\
  - Port par défaut : 80, inutile à préciser
- Indication d'une méthode dans la requête
  - GET : récupérer des informations
  - POST : envoyer des informations pour créer une ressources
  - PUT : envoyer des informations pour modifier entièrement une ressource
  - DELETE : supprimer des informations
  - Et quelques autres…
- Des paramètres (pour la connexion, l'hôte, les cookies)
- Des données (texte), selon des formats divers...
  - Par exemple : param1=val1&param2=val2&…
    - Au bout de l'URI si méthode GET (avec « ? » pour délimiter)
    - Dans le contenu du message si méthode POST ou PUT

Exemple de requête HTTP

```
POST /chemin/ressource HTTP/1.1
Host: www.nom_serveur.com
Connection: Close
Content-type: application/x-www-form-urlencoded
Content-Length: 19

var1=val1&var2=val2
```

- Méthode : POST
- Resource : *chemin/ressource*
- Protocole : HTTP 1.1
- Paramètres
  - Nom du serveur
  - Type de connexion (permanente ou non)
  - Type de contenu (ici, données formatées comme pour une URI)
  - Longueur du contenu (19 caractères)
- Contenu
  - Deux variables

Réponse HTTP

- Information d'état (ok ou erreur)
- Quelques paramètres
  - Type de contenu
    - Dans l'approche de base, page HTML
- Le contenu

Exemple de réponse HTTP

```
HTTP/2 200 OK
Date: Tue, 1 Jan 2019 00:00:01 GMT
Server: Apache/2.0.xx
Connection: close
Content-Type: text/html; charset=UTF-8

<html>
<head>
  <meta http-equiv="content-type" content="text/html; charset=UTF-8" />
  <title>Bonne année</title>
</head>
<body><p>Bonne année!</p></body></html>
```

- Statut : 200 = pas de problème, protocole HTTP 2.0
- Paramètres
  - Date, type de serveur Web, type de connexion, type de contenu
- Contenu
  - Page HTML

Approche « Services Web »

Les services REST

Les services Web

- Plusieurs versions, dont :
  - Premières approches « simplifiée » avec XML
    - XML = langage pour décrire des données avec des balises
  - SOAP : données véhiculées en XML, mieux structuré et plus souple mais HTTP utilise juste pour le « transport »
    - SOAP = Simple Object Access Protocol
    - Plutôt orienté « service »
  - REST : plus simple à mettre en œuvre, exploite plus les services déjà offerts par HTTP
    - REST = REpresentational State Transfer
    - Plutôt orienté « ressource »

Services REST

- A l'origine, approche REST plutôt orientées ressources
  - En lien avec une base de données
- Services attendus
  - Enregistrer de nouvelles données
  - Consulter des données
  - Modifier des données
  - Supprimer des données
  - En résumé : CRUD = Create, Retreive, Update, Delete
  - Méthodes respectives : POST, GET, PUT, DELETE
- URI : localiser la ressource
  - Ex : GET /livres/drame/les_miserables
  - Ex : GET /livres/drame/les_miserables/auteur

Données en services REST

- Paramètres fournis au serveur (par exemple lors d'une création)
  - Envoyés en donnée dans le contenu de la requête car méthode PUT (ou POST)
    - Format URI : varnum=valeur&vartexte=texte&…
    - Format XML
    - Format JSON : { "varnum" : valeur, "vartexte" : "texte" }

  Choix possible grâce au « type de contenu » de l'entête HTTP
- Résultat (par exemple lors d'une consultation)
  - Envoyé en données dans la réponse HTTP
  - En texte, en XML, en JSON, selon le type de contenu de la réponse
  - Champ « accept » de l'entête de la requête pour savoir quels types de contenu autorisés

Exemple de requêtes et réponses

- Requête

```
GET /livres/drame/les_Miserables/auteur HTTP/1.1
Host: www.mabibliotheque.fr
Connection: Close
Content-Length: 0
```

- Réponse

```
HTTP/1.1 200 OK
Server: Apache/2.0.xx
Connection: close
Content-Type: application/json

{ "auteur" : "Victor Hugo" }
```

Discussion sur l'approche HTTP+REST

- Avantages
  - Purement texte, compatible environnements hétérogènes
  - Structure des données envoyées : sémantique+valeur
- Inconvénient
  - Protocole plus ou moins verbeux selon les codages d'information utilisés (XML = lourd, JSON= raisonnable)
  - Basé sur TCP (protocole lourd si contraintes énergétiques)

- Adaptation de l'approche HTTP+REST aux réseaux de capteurs =CoAP (Constrained Application Protocol)
  - Utilisation de 6LoWPAN comme couche réseau
  - Utilisation d'UDP (et non TCP) comme couche transport (et DTLS pour sécuriser par chiffrement sur UDP)
  - CoAP = surcouche d'UDP pour le fiabiliser (mécanisme de ré-ordonnancement, acquittement & ré-expédition)

Format de message CoAP

- Les champs de l'entête CoAP
  - 2 bits de version
  - 2 bits de type de message
    - 00 = confirmable
    - 01 = non confirmable
    - 10 = ACK
    - 11 = reset
  - 4 bits de TKL (Token length) = taille du champ token (à suivre)
  - 8 bits de code « c.dd »,
    décomposé en 3 bits de classe (c) puis 5 bits de détails (dd)
    - Classe = 0 : requête (dd indique la méthode GET, POST, etc.)
    - Classe = 2 : réponse OK
    - Classe = 4/5 : erreur client/serveur
  - 16 bits : Identifiant du message (pour permettre les ACK)
  - 0 à 8 octets : token (pour association requête/réponse)

Approche MQTT

MQTT

- MQTT = *Message Queuing Telemetry Transport*
  - Protocole simple et léger, de couche Session, au dessus de TCP
  - Pas de prise en charge du format des données (c.f. Présentation)
- Système de transmission de messages
  - Système de type « Publish/subscribe »(publication/souscription)
  - Pas de couplage direct producteur/consommateur des données
  - Système d'abonnement = gestion souple de la réception (y compris multi-récepteurs)
- Entités
  - 1 message broker (= le serveur)
  - Un à plusieurs clients connectés au broker
- Principe
  - 1 client envoie un message au broker dans un topic donné
  - Le broker renvoie ce message à tous les clients abonnés au topic

Messages de MQTT

- 4 types de messages
  - Connect : Message d'abonnement (à un topic)
  - Disconnect : Message de désabonnement
  - Publish : Envoi de données (256Mo max) dans deux cas
    - Client producteur vers broker
    - Broker vers clients abonnés
- 2 caractéristiques à l'envoi de donnée
  - Le topic d'appartenance du messages
  - La qualité de service
    - At most once : qualité de base
    - At least once : tout est fait pour faire parvenir un message, avec ré-expédition si pas d'acquittement
    - Exactly once : dialogue plus précis pour garantir la livraison avec ré-expédition seulement si nécessaire
- Aucune gestion de la sécurité mais utilisation de SSL au dessus de TCP possible

Conclusion : CoAP vs MQTT

- Entête du protocole
  - Taille variable de quelques octets dans les deux cas
- Nombre de messages
  - MQTT basé sur TCP ⇨ Lourd
  - CoAP basé sur UDP avec son propre système de fiabilisation
  - ↳ Moins de messages avec CoAP, donc plus économe que MQTT
- Contenu des messages
  - MQTT : contenu décidé en couche présentation, format binaire (plus compact) possible
  - CoAP : plutôt ASCII (XML , JSON…) car issu de REST
    - Compression possible (format EXI=Efficient XML Interchange)
- Utilisatoin
  - Principe du « publish/subscribe » de MQTT plus souple